

Jm G. Pintin | 23-1-00073

Fundamentals of Database Systems

Dr. Jonah Flor O. Maaghop

Visayas State University
Student Records Management System
Using Python and MySQL

November 15, 2024

TABLE OF CONTENTS

	Page
ABSTRACT.....	3
INTRODUCTION.....	3
SYSTEM DESIGN.....	3
SYSTEM COMPONENTS.....	4
GUI.....	4
DATABASE.....	12
DATABASE CONNECTIVITY.....	12
FUNCTIONALITIES.....	13
ADDING STUDENTS FUNCTION PLAYTHROUGH.....	13
EDITING STUDENTS' INFO FUNCTION PLAYTHROUGH.....	16
CLEAR FUNCTION PLAYTHROUGH.....	19
DELETE STUDENT INFO FUNCTION PLAYTHROUGH.....	20
VIEW STUDENT INFO FUNCTION PLAYTHROUGH.....	23
SORT STUDENT INFO FUNCTION PLAYTHROUGH.....	25
SEARCH STUDENT INFO FUNCTION PLAYTHROUGH.....	26
FIXES.....	27
IMPLEMENTATION.....	28
DEVELOPMENT ENVIRONMENT.....	28
GUI DEVELOPMENT.....	28
DATABASE INTERACTIONS.....	28
RESULTS AND DISCUSSION.....	28
CONCLUSION AND FUTURE WORK.....	29
REFERENCES.....	29

ABSTRACT

Student Records Management Systems (SRMS) are essential for managing large volumes of student data efficiently. This laboratory exercise presents the design and implementation of a desktop-based SRMS using Python and MySQL. The system offers functionalities such as adding, updating, deleting, and retrieving student records. It also includes an interactive graphical user interface (GUI) developed with Tkinter, which simplifies user interaction. This discusses the system architecture, implementation, and potential applications in academic institutions.

INTRODUCTION

Efficient management of student data is a critical task for academic institutions. Traditional paper-based record-keeping systems are prone to errors and inefficiencies. This laboratory exercise presents a computerized Student Records Management System that streamlines data management processes. Python, a versatile programming language, and MySQL, a relational database management system, are employed to develop the system.

SYSTEM DESIGN

The proposed system is designed to handle essential student data, including student ID, surname, first name, birth date, gender, and address. The architecture comprises a front-end GUI developed using Python's Tkinter library and a back-end database managed by MySQL.

SYSTEM COMPONENTS

- **GUI** - Designed using Tkinter, the interface allows users to input, view, and modify student data through various widgets such as labels, entry fields, buttons, and combo boxes.

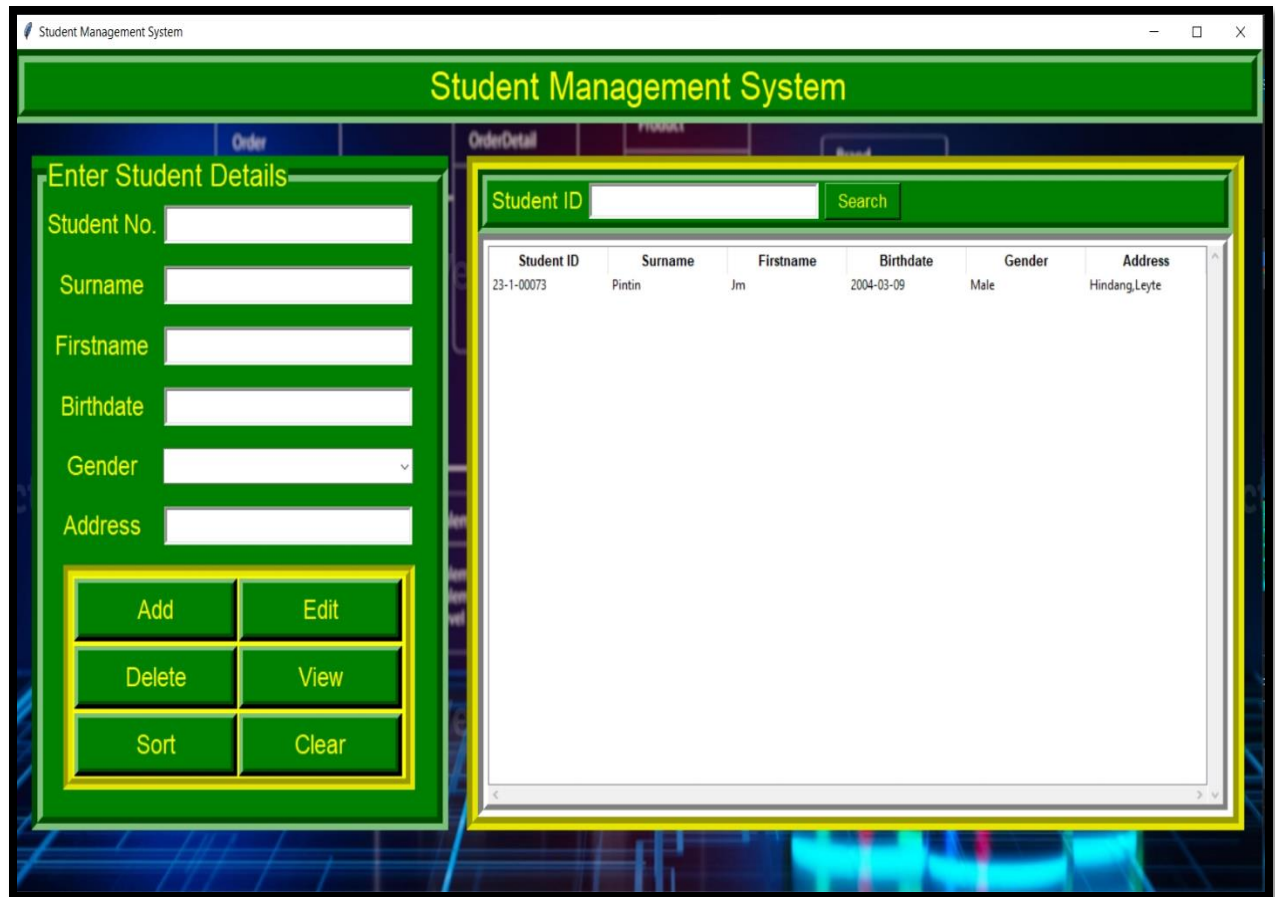


Figure 1. Initial GUI Design of the Main Window

- Initial design of the Main Window. (No functions implemented yet)
- One record is inserted first for testing.
- Inserted an image of the background of the frames.

```

7  win = tk.Tk()
8  win.geometry("1350x750+0+0")
9  win.title("Student Records Management System")
10
11 # Create a Canvas for the background image
12 canvas = tk.Canvas(win, width=1350, height=750)
13 canvas.place(x=0, y=0)
14 #canvas.pack(fill=tk.BOTH, expand=True)
15
16 # Load and display the background image
17 background_image = Image.open("photo.jpg")
18 background_photo = ImageTk.PhotoImage(background_image.resize((1350, 750)))
19 canvas.create_image(0, 0, image=background_photo, anchor=tk.NW)
20
21 title_label = tk.Label(
22     win,
23     text="Student Records Management System",
24     font=("Arial",26),
25     border=12,
26     relief=tk.GROOVE,
27     bg="green",
28     foreground="yellow"
29 )
30
31 title_label.pack(side=tk.TOP,fill = tk.X)
32

```

Figure 2.1. Source Code of GUI Design of the Main Window

```

33 detail_frame = tk.LabelFrame(
34     win,
35     text="Enter Student Details",
36     font=("Arial", 20),
37     bd = 12,
38     relief = tk.GROOVE,
39     bg = "green",
40     fg = "yellow"
41 )
42
43 detail_frame.place(x=20,y=95,width =450,height = 600)
44
45 data_frame = tk.Frame(
46     win,
47     bd = 12,
48     bg = "yellow",
49     relief = tk.GROOVE
50 )
51
52 data_frame.place(x=490,y=95,width = 840,height= 600)
53

```

Figure 2.2. Source Code of GUI Design of the Main Window

```

studentno = tk.StringVar()
surname = tk.StringVar()
firstname = tk.StringVar()
birthdate = tk.StringVar()
gender = tk.StringVar()
address = tk.StringVar()

search_by = tk.StringVar()

#=====#

#INPUTS#

studentno_lbl = tk.Label(
    detail_frame,
    text="Student No.",
    font=("Arial",17),
    bg="green",
    fg="yellow"
)

studentno_lbl.grid(row = 0,column= 0, padx = 2, pady = 10)

```

Figure 2.3. Source Code of GUI Design of the Main Window

```

79 studentno_ent = tk.Entry(
80     detail_frame,
81     bd = 3,
82     font=("Arial",17),
83     textvariable=studentno
84 )
85
86 studentno_ent.grid(row = 0,column= 1, padx = 2, pady = 10)
87
88
89 surname_lbl = tk.Label(
90     detail_frame,
91     text="Surname",
92     font=("Arial",17),
93     bg="green",
94     fg="yellow"
95 )
96
97 surname_lbl.grid(row = 1,column= 0, padx = 2, pady = 10)
98
99 surname_ent = tk.Entry(
00     detail_frame,
01     bd = 3,
02     font=("Arial",17),
03     textvariable=surname
04 )
05
06 surname_ent.grid(row = 1,column= 1, padx = 2, pady = 10)
07

```

Figure 2.4. Source Code of GUI Design of the Main Window

```

107
108     firstname_lbl = tk.Label(
109         detail_frame,
110         text="Firstname",
111         font=("Arial",17),
112         bg="green",
113         fg="yellow"
114     )
115
116     firstname_lbl.grid(row = 2,column= 0, padx = 2, pady = 10)
117
118     firstname_ent = tk.Entry(
119         detail_frame,
120         bd = 3,
121         font=("Arial",17),
122         textvariable=firstname
123     )
124
125     firstname_ent.grid(row = 2,column= 1, padx = 2, pady = 10)
126
127
128     birthdate_lbl = tk.Label(
129         detail_frame,
130         text="Birthdate",
131         font=("Arial",17),
132         bg="green",
133         fg="yellow"
134     )
135
136     birthdate_lbl.grid(row = 3,column= 0, padx = 2, pady = 10)
137

```

Figure 2.5. Source Code of GUI Design of the Main Window

```

136     birthdate_lbl.grid(row = 3,column= 0, padx = 2, pady = 10)
137
138     birthdate_ent = tk.Entry(
139         detail_frame,
140         bd = 3,
141         font=("Arial",17),
142         textvariable=birthdate
143     )
144
145     birthdate_ent.grid(row = 3,column= 1, padx = 2, pady = 10)
146
147     gender_lbl = tk.Label(
148         detail_frame,
149         text="Gender",
150         font=("Arial",17),
151         bg="green",
152         fg="yellow"
153     )
154
155     gender_lbl.grid(row = 4,column= 0, padx = 2, pady = 10)
156
157     gender_ent = ttk.Combobox(
158         detail_frame,
159         #bd = 3,
160         font=("Arial",17),
161         width=19, # Adjust width as needed
162         state="readonly",
163         textvariable= gender
164     )
165

```

Figure 2.6. Source Code of GUI Design of the Main Window


```

166 gender_ent['values'] = ("Male", "Female")
167 gender_ent.grid(row = 4, column= 1, padx = 2, pady = 10)
168
169 address_lbl = tk.Label(
170     detail_frame,
171     text="Address",
172     font=("Arial",17),
173     bg="green",
174     fg="yellow"
175 )
176
177 address_lbl.grid(row = 5, column= 0, padx = 2, pady = 10)
178
179 address_ent = tk.Entry(
180     detail_frame,
181     bd = 3,
182     font=("Arial",17),
183     textvariable=address
184 )
185
186 address_ent.grid(row = 5, column= 1, padx = 2, pady = 10)
187
188 #=====#
189

```

Figure 2.7. Source Code of GUI Design of the Main Window

- Figures 2.1 – 2.7 are the source code for implementing the basic designs of the GUI
- 2 Frames in making the GUI namely detail_frame (left) and data_frame (right).
- These are the Student ID labels, Surname, Firstname, Birthdate, Gender, and Address labels with input box beside it.

```

#buttons

btn_frame = tk.Frame(
    detail_frame,
    bg = "yellow",
    bd =10,
    relief = tk.GROOVE
)

btn_frame.place(x = 22, y = 330, width = 380, height = 200)

add_btn = tk.Button(
    btn_frame,
    text="Add",
    font=("Arial",17),
    bg="green",
    fg = "yellow",
    bd = 7,
    width = 12,
    command=add_func
)

add_btn.grid(row=0, column=0, padx =2 , pady = 2)

```

Figure 3. Source Code of GUI Design of the Main Window (Buttons)


```

edit_btn = tk.Button(
    btn_frame,
    text="Edit",
    font=("Arial",17),
    bg="green",
    fg = "yellow",
    bd = 7,
    width = 12,
    command=edit_func
)

edit_btn.grid(row=0,column=1,padx =0 , pady = 0)

delete_btn = tk.Button(
    btn_frame,
    text="Delete",
    font=("Arial",17),
    bg="green",
    fg = "yellow",
    bd = 7,
    width = 12,
    command=delete_func
)

delete_btn.grid(row=1,column=0,padx =2 , pady = 2)

```

Figure 3.1. Source Code of GUI Design of the Main Window (Buttons)

```

view_btn = tk.Button(
    btn_frame,
    text="View",
    font=("Arial",17),
    bg="green",
    fg = "yellow",
    bd = 7,
    width = 12,
    command=view_func
)

view_btn.grid(row=1,column=1,padx =0 , pady = 0)

sort_btn = tk.Button(
    btn_frame,
    text="Sort",
    font=("Arial",17),
    bg="green",
    fg = "yellow",
    bd = 7,
    width = 12,
    command=sort_func
)

sort_btn.grid(row=2,column=0,padx =2 , pady = 2)

```

Figure 3.2. Source Code of GUI Design of the Main Window (Buttons)

```

478 clear_btn = tk.Button(
479     btn_frame,
480     text="Clear",
481     font=("Arial",17),
482     bg="green",
483     fg = "yellow",
484     bd = 7,
485     width = 12,
486     command=clear
487 )
488
489 clear_btn.grid(row=2,column=1,padx =0 , pady = 0)
490
491 #searchframe#
492
493 search_frame = tk.Frame(
494     data_frame,
495     bg = "green",
496     bd = 10,
497     relief = tk.GROOVE
498 )
499
500 search_frame.pack(
501     side=tk.TOP,
502     fill=tk.X
503 )
504

```

Figure 3.3. Source Code of GUI Design of the Main Window (Buttons)

```

505 search_lbl = tk.Label(
506     search_frame,
507     text="Student ID",
508     bg = "green",
509     fg = "yellow",
510     font=("Arial",16)
511 )
512
513 search_lbl.grid(row=0,column=0,padx=2,pady=2)
514
515 search_in = tk.Entry(
516     search_frame,
517     font=("Arial",16),
518     bd = 3,
519     textvariable=search_by
520 )
521
522 search_in.grid(row=0, column = 1, padx = 2, pady = 2)
523
524 search_but = tk.Button(
525     search_frame,
526     text="Search",
527     font=("Arial",13),
528     bg="green",
529     fg = "yellow",
530     bd = 2,
531     width = 8,
532     command=search_func
533 )
534
535 search_but.grid(row =0, column = 2, padx = 5, pady = 2)
536

```

Figure 3.4. Source Code of GUI Design of the Main Window (Buttons)

- Figures 3 – 3.4 are the source code for implementing the basic designs of the GUI Buttons.
- These are the Add, Edit, Delete, Sort, View, Clear and Search.

```

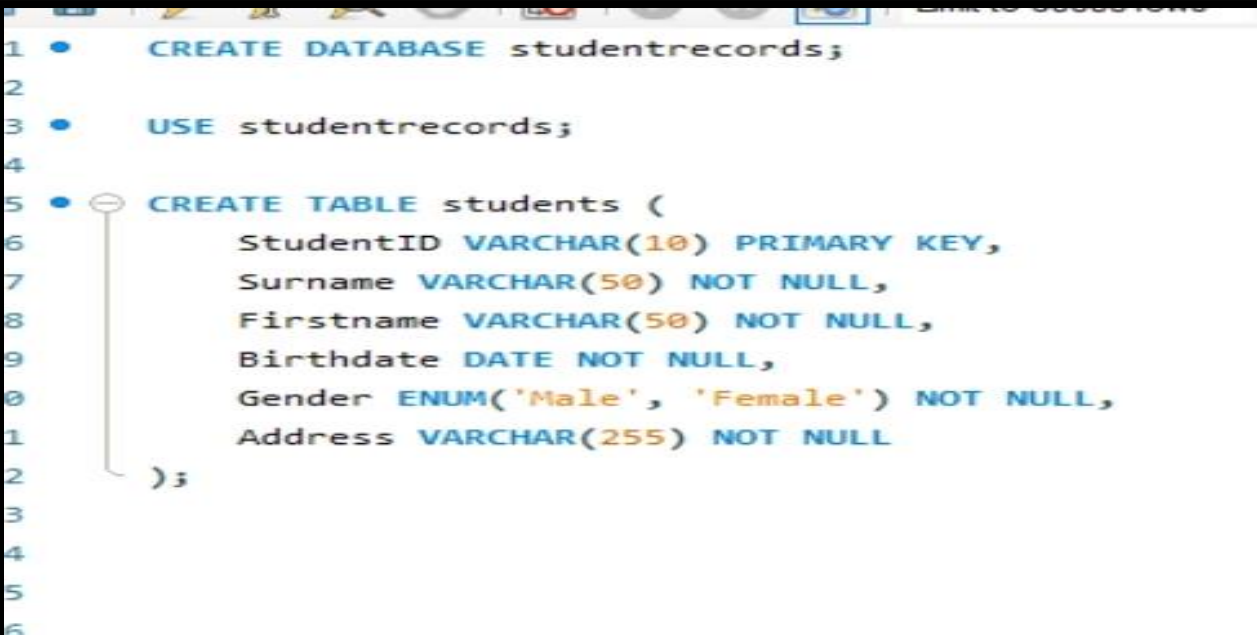
537 #Main data frame
538
539 style = ttk.Style()
540 style.configure("Treeview.Heading", font=("Arial", 10, "bold")) # Set bold font for column headings
541
542 # Main data frame
543 main_frame = tk.Frame(data_frame, bg="lightgrey", bd=11, relief=tk.GROOVE)
544 main_frame.pack(fill=tk.BOTH, expand=True)
545
546 y_scroll = tk.Scrollbar(main_frame, orient=tk.VERTICAL)
547 x_scroll = tk.Scrollbar(main_frame, orient=tk.HORIZONTAL)
548
549 student_table = ttk.Treeview(
550     main_frame,
551     columns=("Student ID", "Surname", "Firstname", "Birthdate", "Gender", "Address"),
552     yscrollcommand=y_scroll.set,
553     xscrollcommand=x_scroll.set,
554 )
555
556 y_scroll.config(command=student_table.yview)
557 x_scroll.config(command=student_table.xview)
558
559 y_scroll.pack(side=tk.RIGHT, fill=tk.Y)
560 x_scroll.pack(side=tk.BOTTOM, fill=tk.X)
561
562 student_table.heading("Student ID", text="Student ID")
563 student_table.heading("Surname", text="Surname")
564 student_table.heading("Firstname", text="Firstname")
565 student_table.heading("Birthdate", text="Birthdate")
566 student_table.heading("Gender", text="Gender")
567 student_table.heading("Address", text="Address")
568
569 student_table["show"] = "headings"
570
571 student_table.column("Student ID", width = 100)
572 student_table.column("Surname", width = 100)
573 student_table.column("Firstname", width = 100)
574 student_table.column("Birthdate", width = 100)
575 student_table.column("Gender", width = 100)
576 student_table.column("Address", width = 100)
577
578 student_table.pack(fill=tk.BOTH, expand=True)
579

```

Figure 4. Source Code of GUI Design of Student Table Record

- Treeview where we can see the inputted data from the database.
- Right side of the GUI.

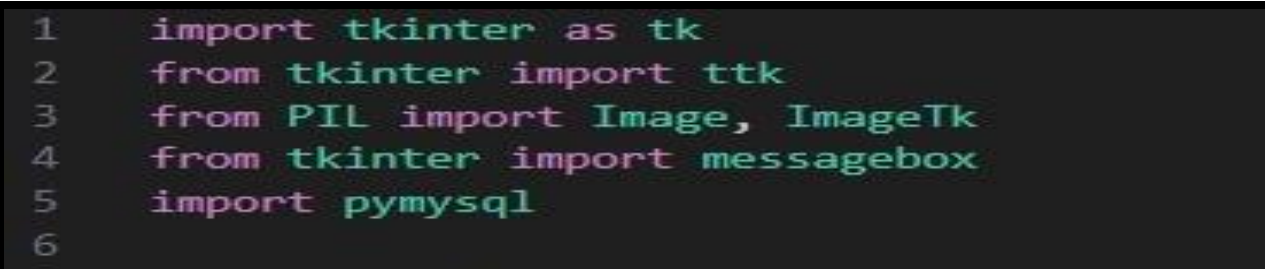
- **DATABASE** - The MySQL database contains a single table, students, structured as follows:



```
1 • CREATE DATABASE studentrecords;
2
3 • USE studentrecords;
4
5 • CREATE TABLE students (
6     StudentID VARCHAR(10) PRIMARY KEY,
7     Surname VARCHAR(50) NOT NULL,
8     Firstname VARCHAR(50) NOT NULL,
9     Birthdate DATE NOT NULL,
10    Gender ENUM('Male', 'Female') NOT NULL,
11    Address VARCHAR(255) NOT NULL
12 );
```

Figure 4. SQL Schema

- **DATABASE CONNECTIVITY** – Connect python to MySQL database using these following imports:



```
1 import tkinter as tk
2 from tkinter import ttk
3 from PIL import Image, ImageTk
4 from tkinter import messagebox
5 import pymysql
6
```

Figure 5. Python Imports

- Connect thru `pymysql.connect(...)` .
- `pymysql` is used to connect to a MySQL database.

FUNCTIONALITIES

The system provides the following key functionalities:

1. **Add Record:** Allows users to insert a new student record into the database.
2. **Update Record:** Enables modification of existing records.
3. **Delete Record:** Removes selected records permanently.
4. **Search Record:** Retrieves specific records based on student ID.
5. **Sort Records:** Displays all records sorted by student ID or surname.
6. **View All Records:** Displays all records in a new window using a tabular format.

ADDING STUDENTS FUNCTION PLAYTHROUGH

```
#Functions

def fetch_data():
    conn = pymysql.connect(host="localhost", user="root", password="", database="studentrecords")
    curr = conn.cursor()
    # Fetch data sorted by surname
    curr.execute("SELECT * FROM students ORDER BY surname ASC")
    rows = curr.fetchall()
    if len(rows) != 0:
        student_table.delete(*student_table.get_children())
        for row in rows:
            student_table.insert('', tk.END, values=row)
    conn.close()
```

Figure 6. Fetch data Function Source Code

- Retrieves and displays student details from the database or file.
- This function is often triggered when the user wants to **search for specific students, view all student records, or verify existing data.**


```

def add_func():
    if studentno.get() == "" or surname.get() == "" or firstname.get() == "" or birthdate.get() == "" or gender.get() == "" or address.get() == "":
        messagebox.showerror("Error!", "Please fill all of the fields")
    else:
        try:
            conn = pymysql.connect(host="localhost", user="root", password="", database="studentrecords")
            curr = conn.cursor() # Use a cursor object for executing SQL commands
            curr.execute(
                "INSERT INTO students VALUES (%s, %s, %s, %s, %s, %s)",
                (studentno.get(), surname.get(), firstname.get(), birthdate.get(), gender.get(), address.get())
            )
            conn.commit()
            conn.close()
            messagebox.showinfo("Success", "Student added successfully!")
            fetch_data() # Refresh the table after adding a new student
        except Exception as e:
            messagebox.showerror("Error", f"An error occurred: {e}")

```

Figure 7. Add Student Function Source Code

- Validates input fields before inserting a new student into the database.
- Executes an SQL INSERT command.
- Refreshes the display using fetch_data() and shows a success message.
- **Error Handling:** Displays a meaningful error message if an exception occurs

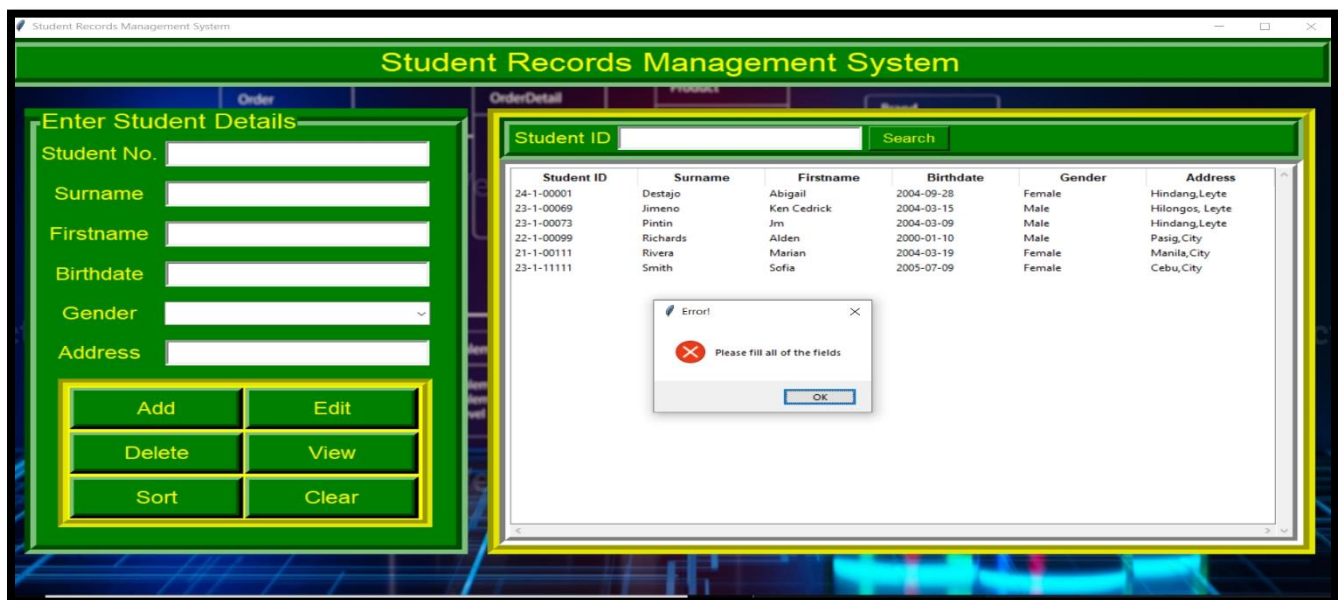


Figure 8. Add Student Button Playthrough

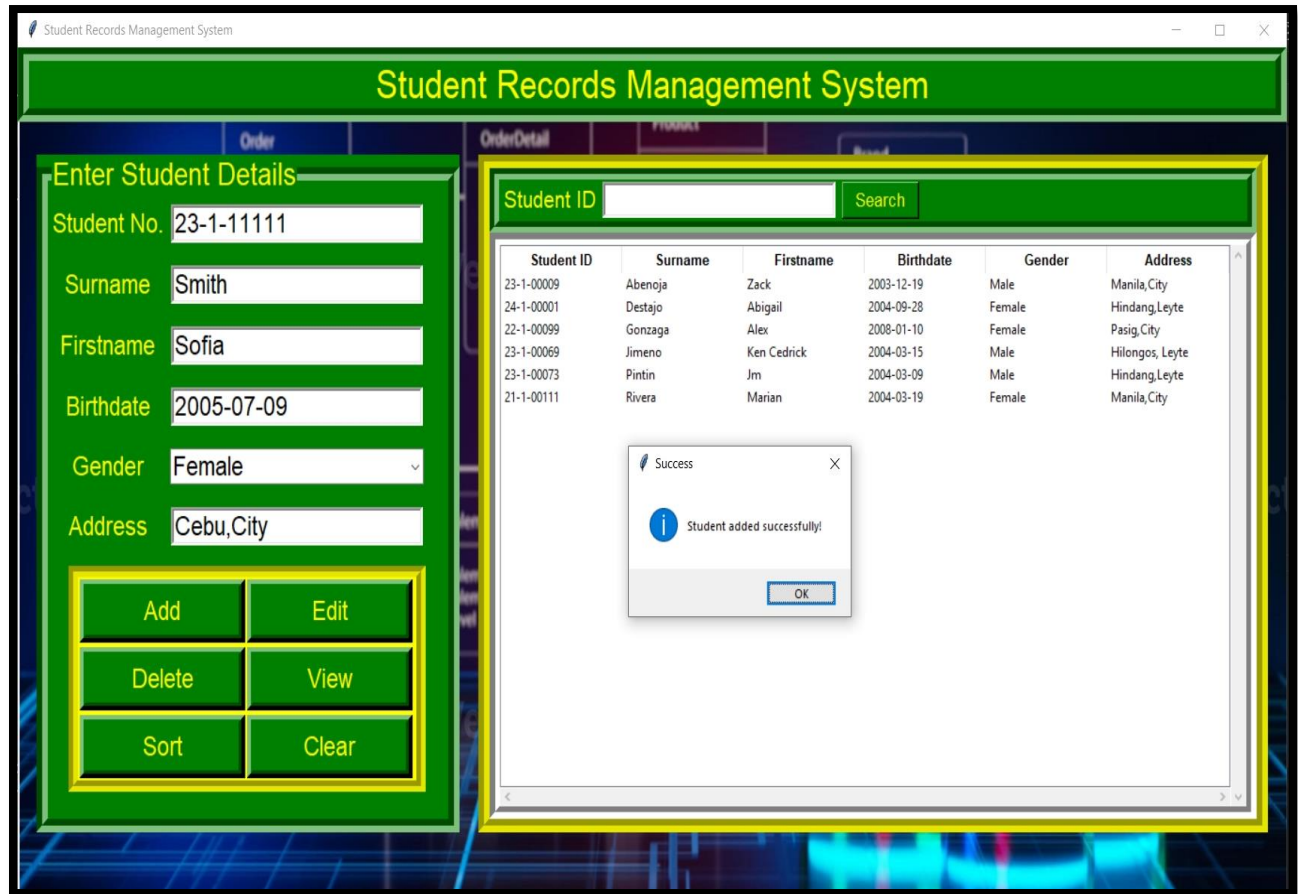


Figure 8.1. Add Student Button Playthrough

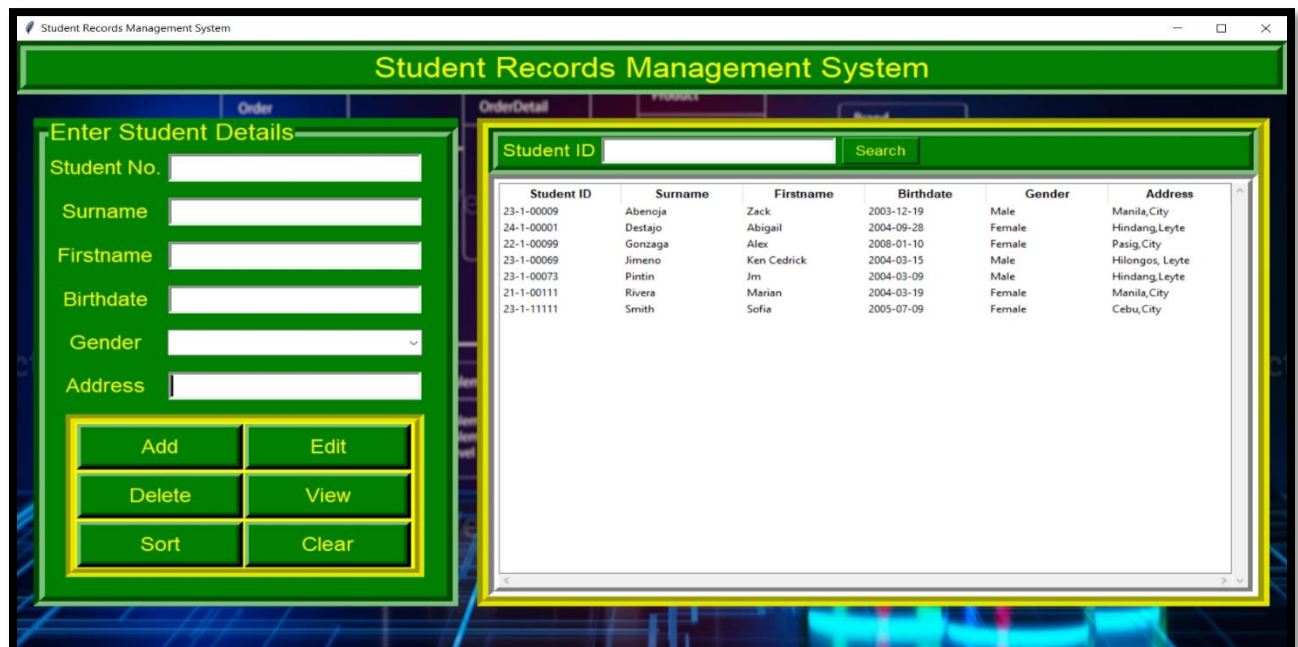


Figure 8.2. Add Student Button Playthrough

EDITING STUDENTS INFO FUNCTION PLAYTHROUGH

```
def get_cursor(event):  
    # Fetch the data of the selected row for edit  
    cursor_row = student_table.focus()  
    content = student_table.item(cursor_row)  
    row = content['values']  
    studentno.set(row[0])  
    surname.set(row[1])  
    firstname.set(row[2])  
    birthdate.set(row[3])  
    gender.set(row[4])  
    address.set(row[5])
```

Figure 9. Get Cursor Function Source Code

- Retrieves and displays data of a selected row from the TreeView in the input fields.
- Triggered by a user selecting a record in the table.
- **Use Case:** Loading details of a student for editing and deletion of records.

```
def edit_func():  
    if studentno.get() == "" or surname.get() == "" or firstname.get() == "" or birthdate.get() == "" or gender.get() == "" or address.get() == "":  
        messagebox.showerror("Error!", "Please select a student to edit.")  
        return  
  
    try:  
        conn = pymysql.connect(host="localhost", user="root", password="", database="studentrecords")  
        curr = conn.cursor()  
        curr.execute(  
            "UPDATE students SET surname=%s, firstname=%s, birthdate=%s, gender=%s, address=%s WHERE StudentID=%s",  
            (surname.get(), firstname.get(), birthdate.get(), gender.get(), address.get(), studentno.get())  
        )  
        conn.commit()  
        conn.close()  
  
        messagebox.showinfo("Success", "Student details updated successfully!")  
        fetch_data() # Refresh the table after editing  
        clear() # Added Clear after editing clear the box for adding  
    except Exception as e:  
        messagebox.showerror("Error", f"An error occurred: {e}")
```

Figure 10. Edit Function Source Code

- Updates the selected student's information in the database.
- Executes an SQL UPDATE query with the provided input data.
- Refreshes the table using fetch_data() and clears the input fields.
- **Validation:** Ensures all fields are filled before updating.

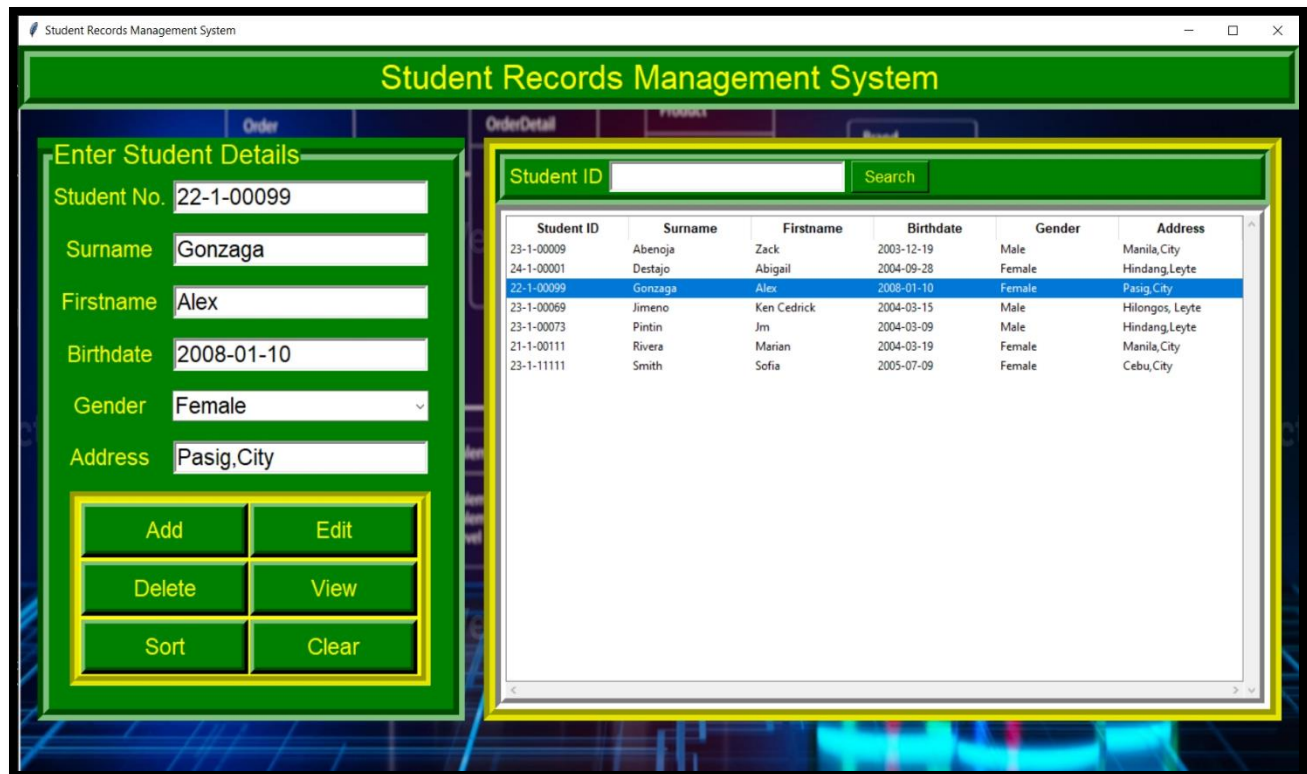


Figure 11. Edit Student Info Button Playthrough

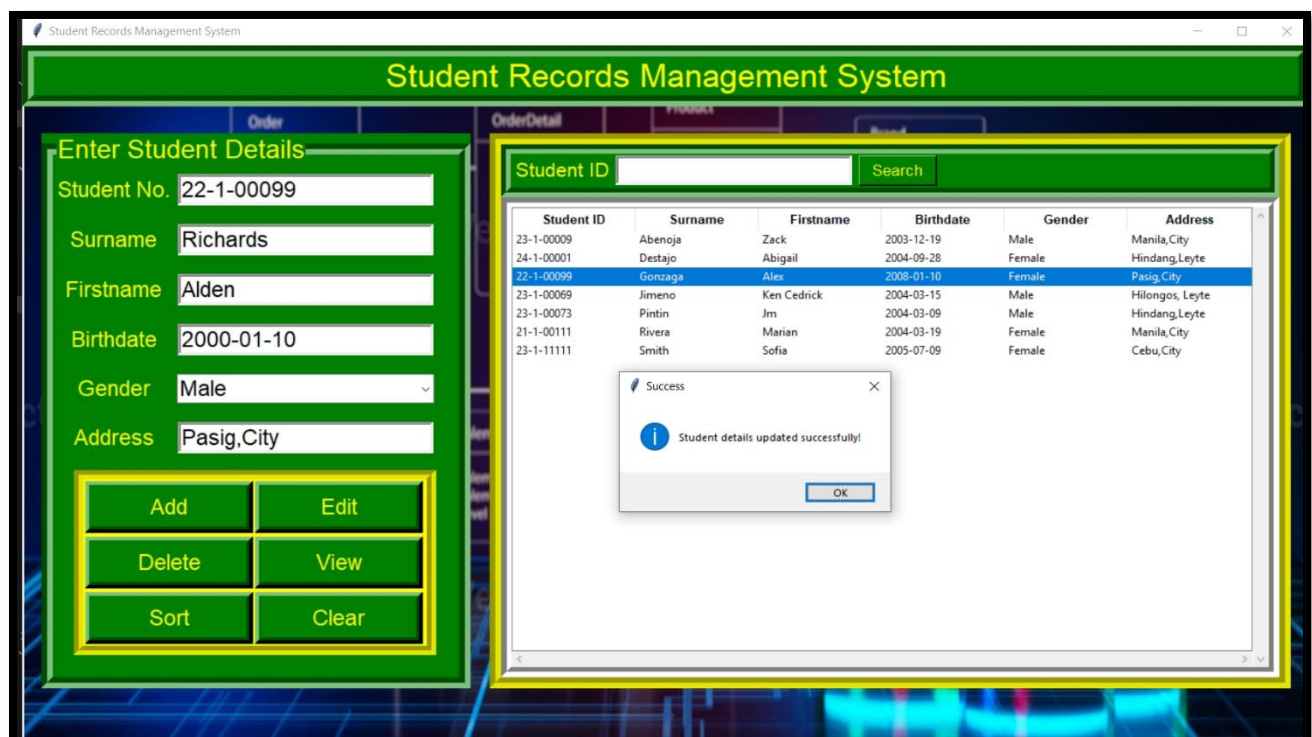


Figure 11.2. Edit Student Info Button Playthrough

The screenshot displays the 'Student Records Management System' interface. On the left, there is a green-bordered form titled 'Enter Student Details' with fields for Student No., Surname, Firstname, Birthdate, Gender (a dropdown menu), and Address. Below these fields are six buttons: Add, Edit, Delete, View, Sort, and Clear. On the right, there is a table of student records with columns for Student ID, Surname, Firstname, Birthdate, Gender, and Address. A search bar with a 'Search' button is located above the table.

Student ID	Surname	Firstname	Birthdate	Gender	Address
23-1-00009	Abenoja	Zack	2003-12-19	Male	Manila, City
24-1-00001	Destajo	Abigail	2004-09-28	Female	Hindang, Leyte
23-1-00069	Jimeno	Ken Cedrick	2004-03-15	Male	Hilongos, Leyte
23-1-00073	Pintin	Jm	2004-03-09	Male	Hindang, Leyte
22-1-00099	Richards	Alden	2000-01-10	Male	Pasig, City
21-1-00111	Rivera	Marian	2004-03-19	Female	Manila, City
23-1-11111	Smith	Sofia	2005-07-09	Female	Cebu, City

Figure 11.3. Edit Student Info Button Playthrough

- Successfully edit the surname, firstname, birthdate and gender of the selected record. (Can edit all if necessary)

CLEAR FUNCTION PLAYTHROUGH

```
def clear():  
    studentno.set("")  
    surname.set("")  
    firstname.set("")  
    birthdate.set("")  
    gender.set("")  
    address.set("")
```

Figure 12. Clear Function Source Code

- Clears all input fields.
- **Use Case:** Prepares fields for a new entry or clears after an operation.

The screenshot shows the 'Student Records Management System' interface. On the left, the 'Enter Student Details' form has fields for Student No., Surname, Firstname, Birthdate, Gender, and Address. Below these fields are buttons for Add, Edit, Delete, View, Sort, and Clear. The 'Clear' button is highlighted with a yellow border. On the right, a table displays student records with columns for Student ID, Surname, Firstname, Birthdate, Gender, and Address. The table contains several rows of data, including students like Zack, Abigail, Ken Cedrick, Jim, Alden, and Sofia.

Figure 13. Clear Function Button Playthrough

This screenshot is identical to the one in Figure 13, showing the 'Student Records Management System' interface. The 'Enter Student Details' form on the left has the 'Clear' button highlighted with a yellow border. The table on the right shows the same student records as in Figure 13.

Figure 12.1. Clear Function Button Playthrough

DELETE STUDENT INFO FUNCTION PLAYTHROUGH

```

264 def delete_func():
265     if studentno.get() == "":
266         messagebox.showerror("Error!", "Please select a student to delete.")
267         return
268
269     confirm = messagebox.askyesno("Confirm Delete", "Are you sure you want to delete this record?")
270     if confirm:
271         try:
272             conn = pymysql.connect(host="localhost", user="root", password="", database="studentrecords")
273             curr = conn.cursor()
274             curr.execute("DELETE FROM students WHERE StudentID = %s", (studentno.get(),))
275             conn.commit()
276             conn.close()
277
278             messagebox.showinfo("Success", "Student record deleted successfully!")
279             fetch_data() # Refresh the table after deletion
280             clear() # Clear the fields after deletion
281         except Exception as e:
282             messagebox.showerror("Error", f"An error occurred: {e}")
283

```

Figure 13. Delete Student Info Function Source Code

- Deletes a selected student record from the database.
- Prompts a confirmation dialog before deletion.
- Executes an SQL DELETE command and refreshes the table.
- **Error Handling:** Prevents deletion if no record is selected.

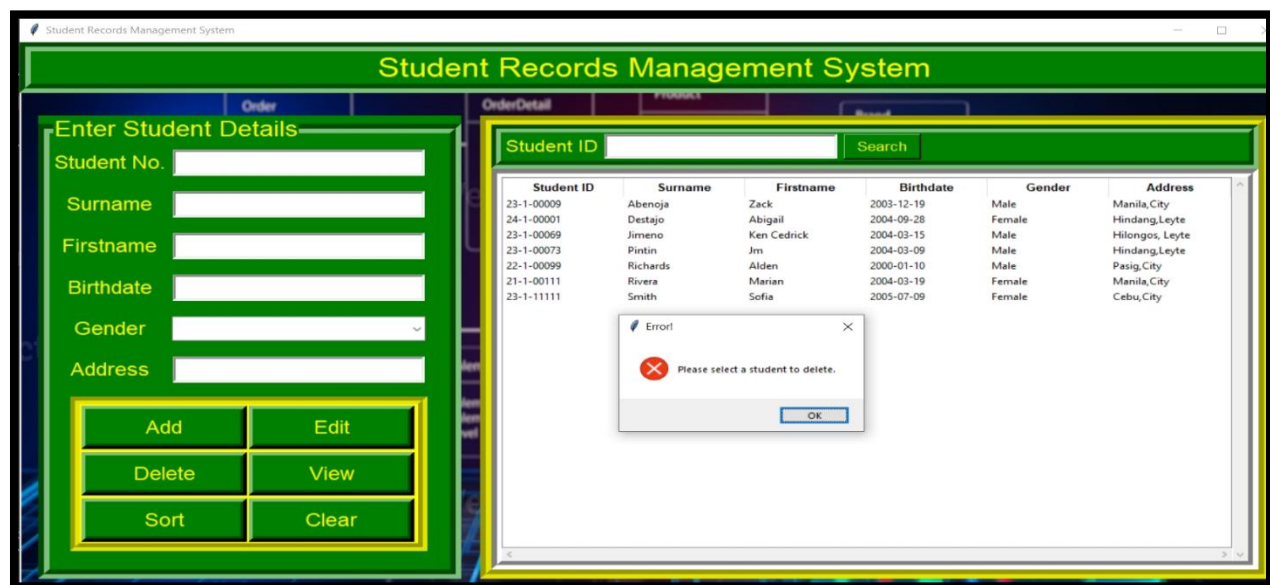


Figure 14. Delete Student Info Button Playthrough

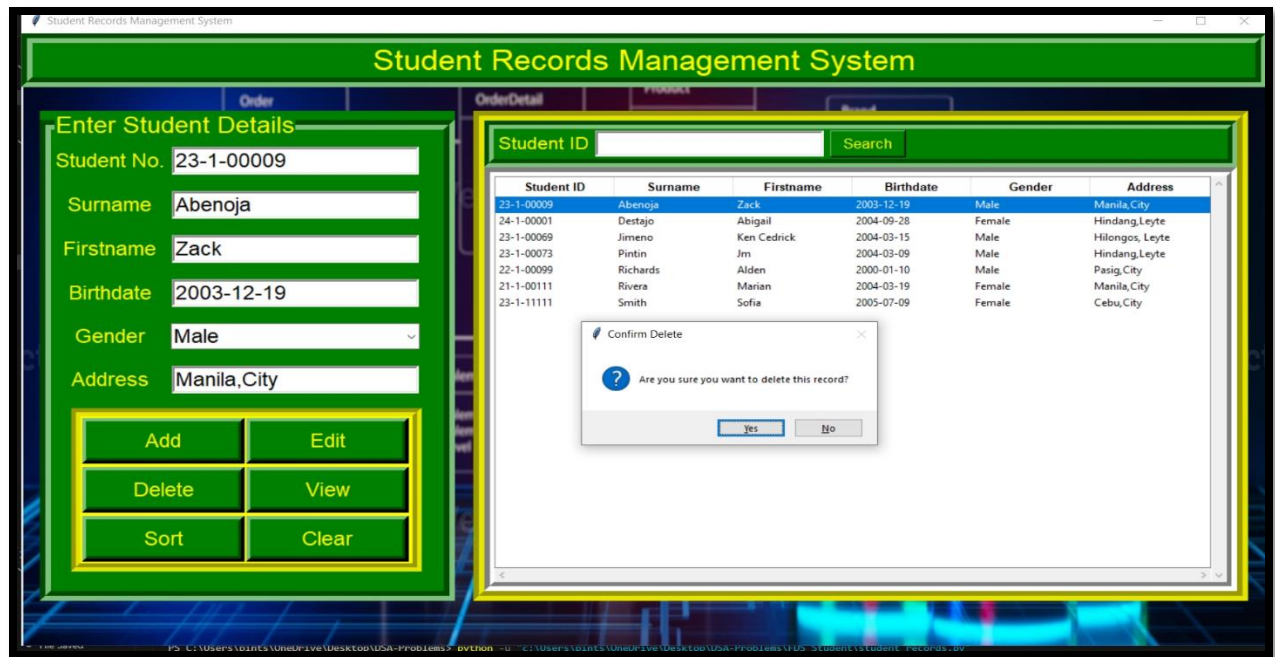


Figure 14.1. Delete Student Info Button Playthrough

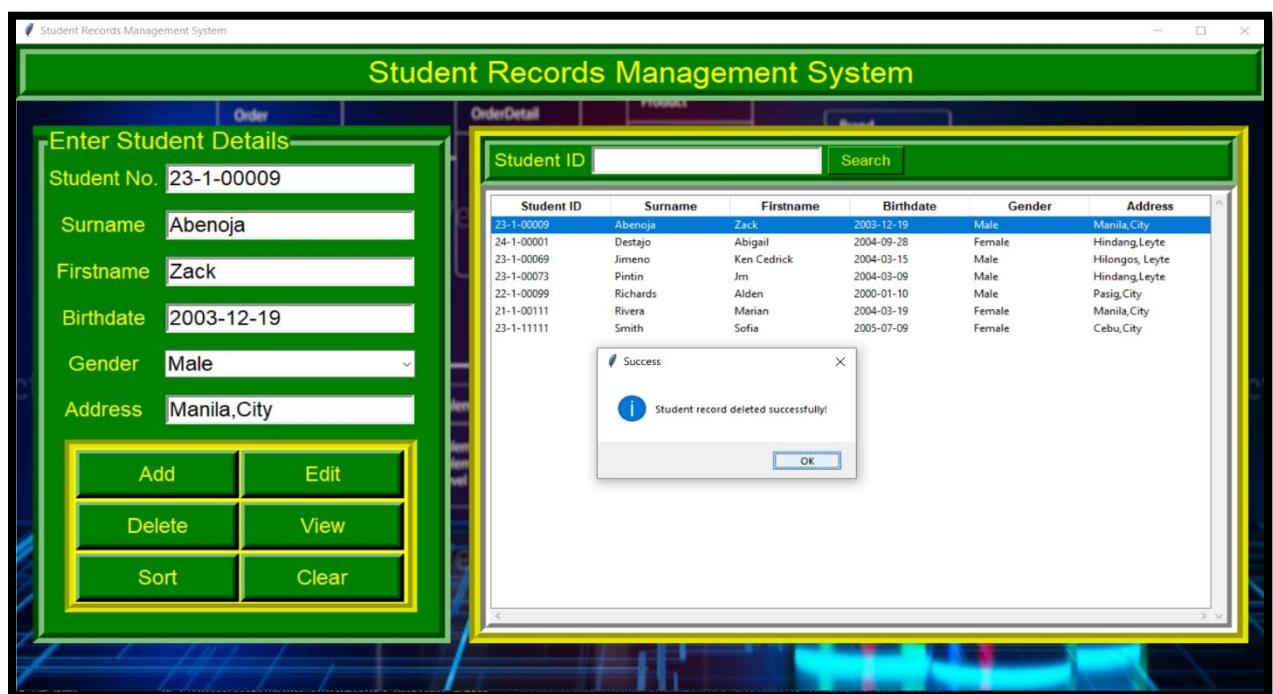


Figure 14.2. Delete Student Info Button Playthrough

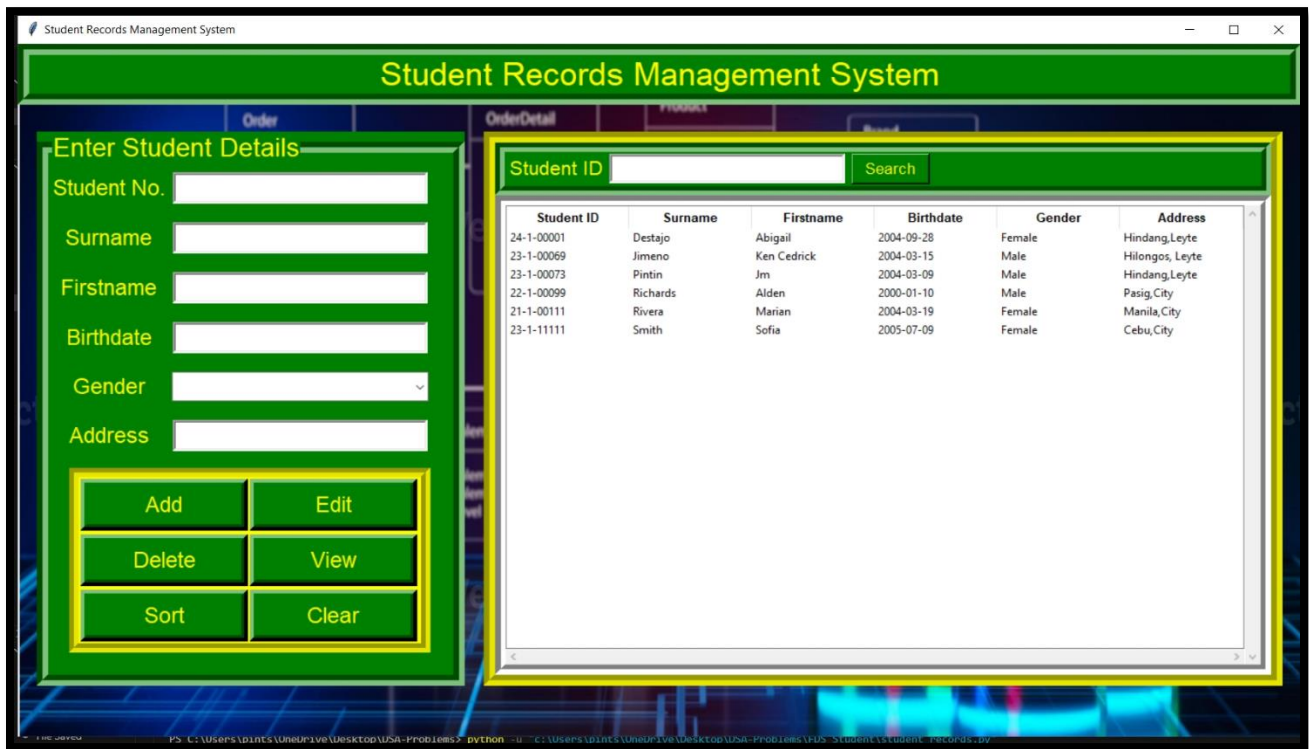


Figure 14.3. Delete Student Info Button Playthrough

- Successfully deleted a record.

VIEW STUDENT INFO FUNCTION PLAYTHROUGH

```
def view_func():
    # Create a new window for viewing student records
    view_window = tk.Toplevel(win)
    view_window.title("Student Records")
    view_window.geometry("900x500")

    # Add a title label
    title = tk.Label(
        view_window,
        text="All Student Records",
        font=("Arial", 22),
        bg="green",
        border=12,
        relief=tk.GROOVE,
        fg="yellow",
        pady=10
    )
    title.pack(fill=tk.X)

    # Create a frame for the Treeview
    frame = tk.Frame(view_window, bd=10, bg="yellow", relief=tk.GROOVE)
    frame.pack(fill=tk.BOTH, expand=True)

    # Add Scrollbars
    y_scroll = tk.Scrollbar(frame, orient=tk.VERTICAL)
    x_scroll = tk.Scrollbar(frame, orient=tk.HORIZONTAL)
```

Figure 15. View Student Info Function Source Code (Separate Window)

```
# Create the Treeview widget
records_table = ttk.Treeview(
    frame,
    columns=("Student ID", "Surname", "Firstname", "Birthdate", "Gender", "Address"),
    yscrollcommand=y_scroll.set,
    xscrollcommand=x_scroll.set
)

y_scroll.config(command=records_table.yview)
x_scroll.config(command=records_table.xview)

y_scroll.pack(side=tk.RIGHT, fill=tk.Y)
x_scroll.pack(side=tk.BOTTOM, fill=tk.X)

records_table.heading("Student ID", text="Student ID")
records_table.heading("Surname", text="Surname")
records_table.heading("Firstname", text="Firstname")
records_table.heading("Birthdate", text="Birthdate")
records_table.heading("Gender", text="Gender")
records_table.heading("Address", text="Address")
records_table['show'] = 'headings'

records_table.column("Student ID", width=100)
records_table.column("Surname", width=150)
records_table.column("Firstname", width=150)
records_table.column("Birthdate", width=100)
records_table.column("Gender", width=100)
records_table.column("Address", width=200)

records_table.pack(fill=tk.BOTH, expand=True)
```

Figure 15.1. View Student Info Function Source Code (Separate Window)

```

# Fetch and display records from the database
try:
    conn = pymysql.connect(host="localhost", user="root", password="", database="studentrecords")
    curr = conn.cursor()
    curr.execute("SELECT * FROM students ORDER BY surname")
    rows = curr.fetchall()

    # Insert all rows into the Treeview
    for row in rows:
        records_table.insert('', tk.END, values=row)

    conn.close()
except Exception as e:
    messagebox.showerror("Error", f"An error occurred while fetching data: {e}")

```

Figure 15.2. View Student Info Function Source Code (Separate Window)

- Opens a new window displaying all student records in a TreeView.
- Fetches data from the database and populates the table.
- **Enhancement:** Provides a quick overview of all records in a separate window.

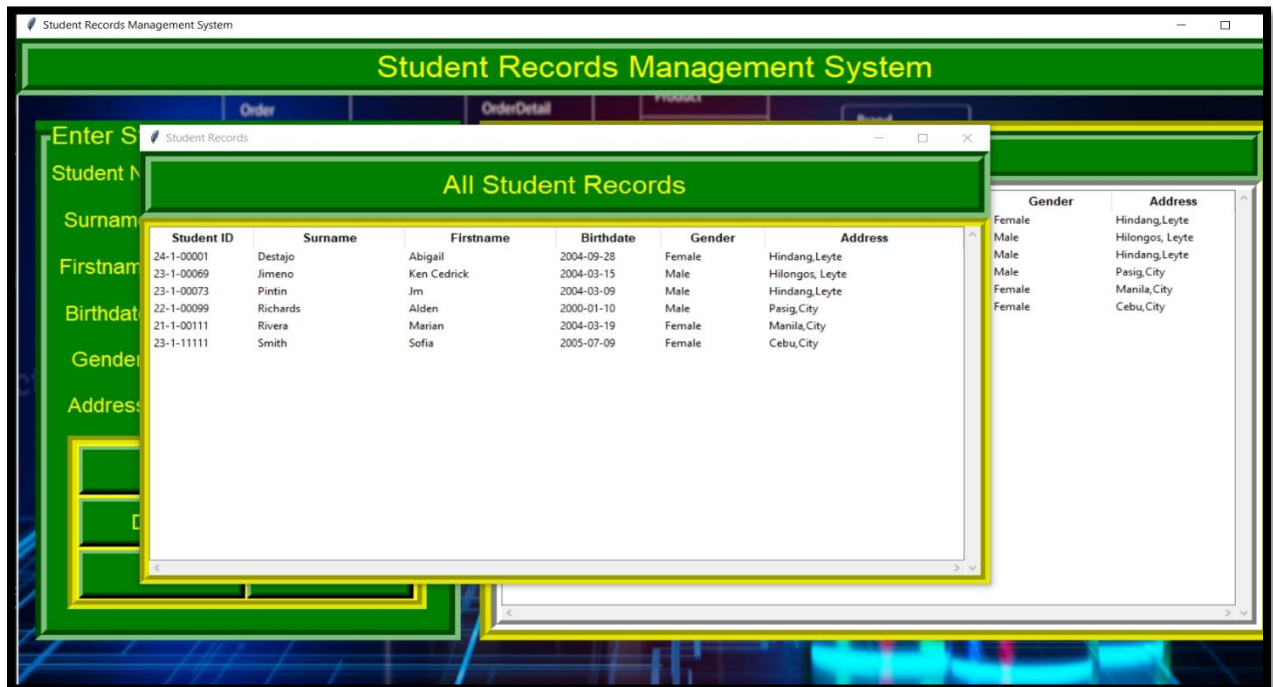


Figure 16. View Student Info Button Playthrough

SORT STUDENT INFO FUNCTION BY STUDENT NO PLAYTHROUGH

```
def sort_func():
    try:
        conn = pymysql.connect(host="localhost", user="root", password="", database="studentrecords")
        curr = conn.cursor()

        # Query to sort by Student ID
        curr.execute("SELECT * FROM students ORDER BY StudentID ASC") # Assuming StudentID is the column name
        rows = curr.fetchall()

        student_table.delete(*student_table.get_children()) # Clear the table before inserting new data
        for row in rows:
            student_table.insert('', tk.END, values=row)

        conn.close()
        # Notify the user that sorting was successful
        messagebox.showinfo("Success", "Records sorted by Student ID successfully!")
    except Exception as e:
        messagebox.showerror("Error", f"An error occurred: {e}")
```

Figure 17. Sort Student Info Function Source Code (by Student ID.)

- Sorts and displays all student records by StudentID in ascending order.
- Executes an SQL ORDER BY query and updates the main table.

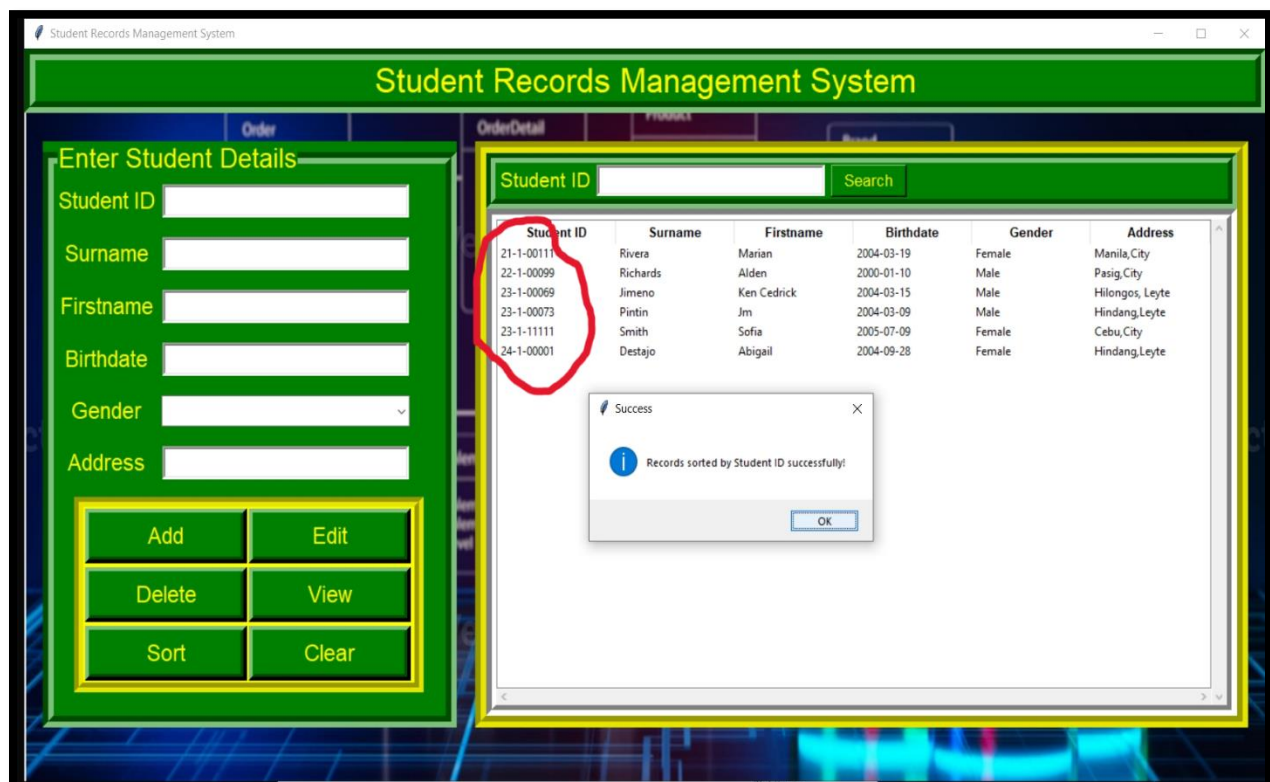


Figure 18. Sort Student Info Button Playthrough (by Student ID.)

SEARCH STUDENT INFO FUNCTION BY STUDENT NO PLAYTHROUGH

```
def search_func():
    if search_by.get() == "":
        # If the search input is empty, fetch all records
        fetch_data() # This will refresh the table with all student records
        return # Exit the function

    try:
        conn = pymysql.connect(host="localhost", user="root", password="", database="studentrecords")
        curr = conn.cursor()
        query = "SELECT * FROM students WHERE StudentID = %s"
        curr.execute(query, (search_by.get(),))
        rows = curr.fetchall()

        if rows:
            student_table.delete(*student_table.get_children())
            for row in rows:
                student_table.insert('', tk.END, values=row)
            messagebox.showinfo("Success", "Search completed successfully!")
        else:
            messagebox.showinfo("No Result", "No student found with the given Student ID.")

        conn.close()
    except Exception as e:
        messagebox.showerror("Error", f"An error occurred: {e}")
```

Figure 19. Search Student Info Function Source Code (by Student ID.)

- Searches for a student by their StudentID and displays the result in the main table.
- If no ID is provided, all records are fetched using fetch_data().

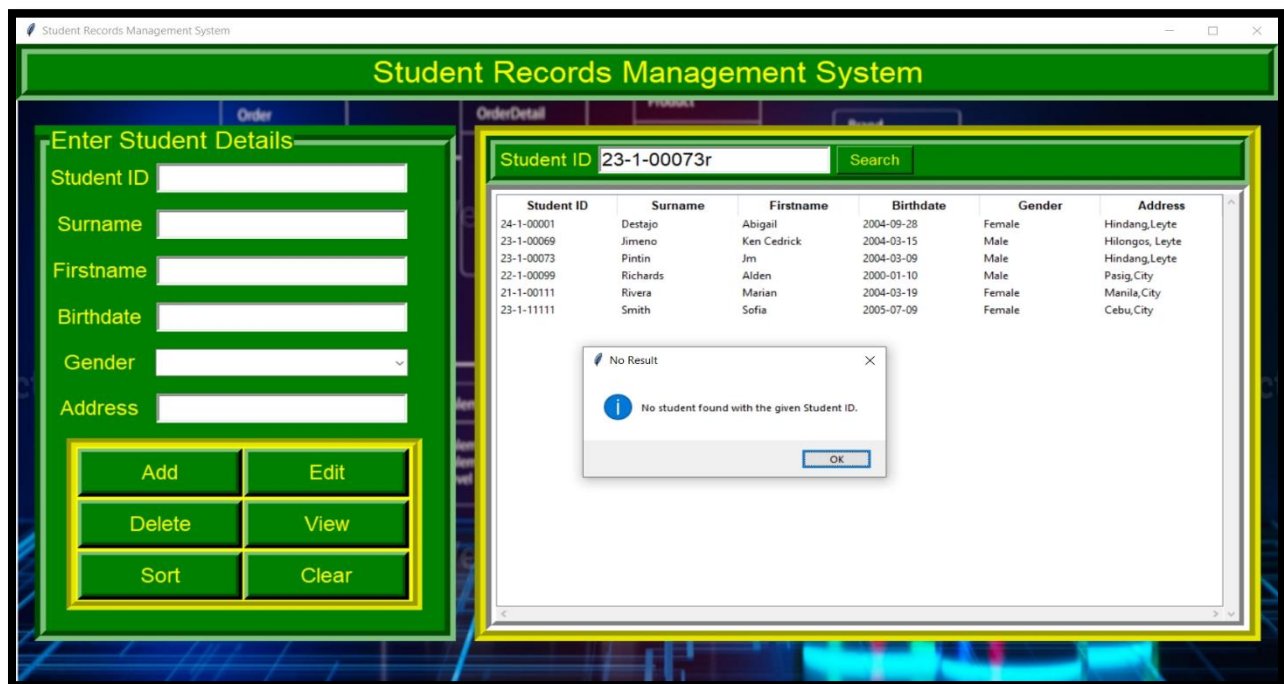


Figure 20. Search Student Info Button Playthrough (by Student ID.)

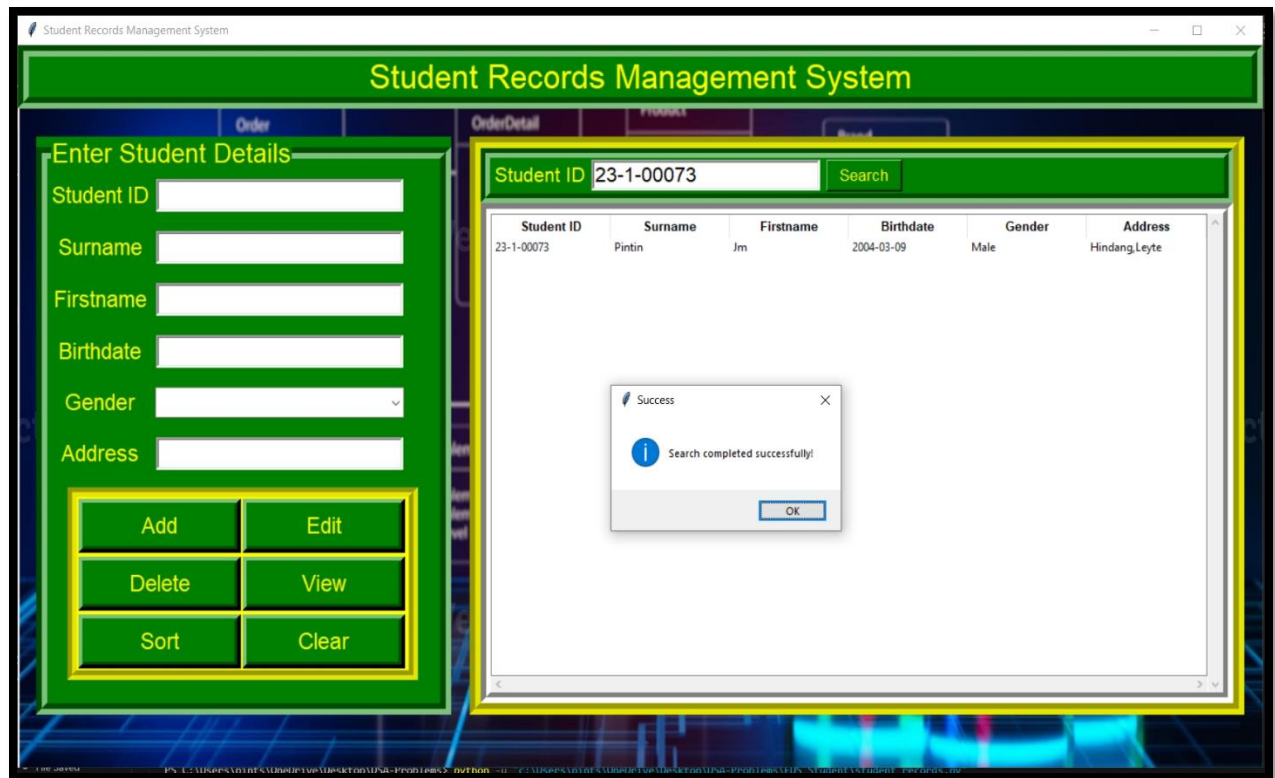


Figure 20.1. Search Student Info Button Playthrough (by Student ID)

FIXES

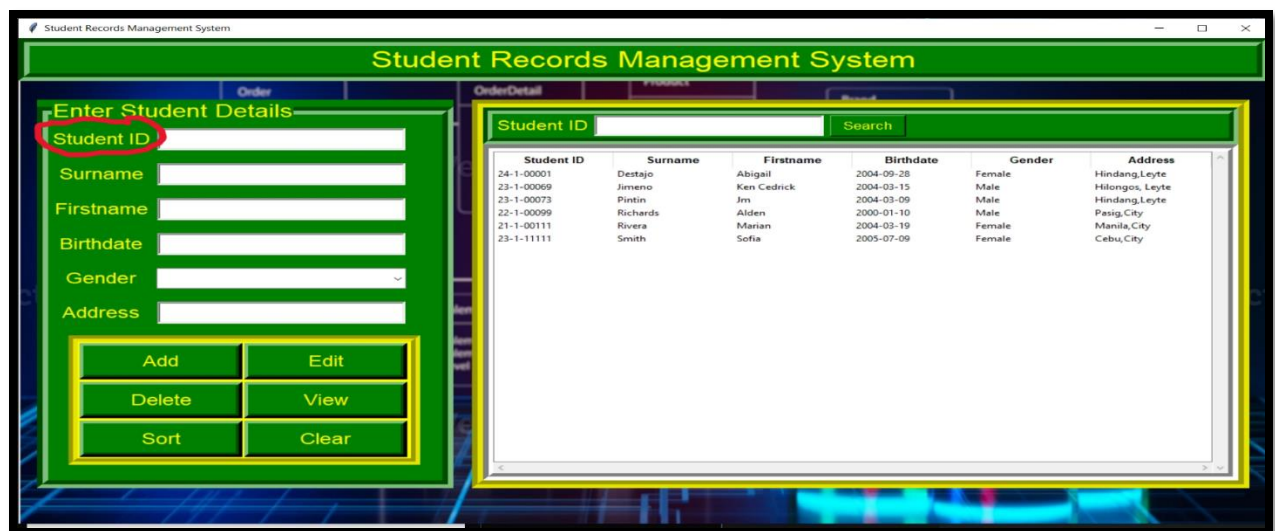


Figure 21: Fixed the Labeling of Student No. to Student ID

- A mistake on my part and realize it along the way.
- Final GUI design.

IMPLEMENTATION

The implementation process involved three key phases:

- **DEVELOPMENT ENVIRONMENT**

- **Programming Language:** Python 3.10
- **Libraries:** Tkinter (GUI), pymysql (database connectivity), PIL (image handling)
- **Database:** MySQL
- **IDE:** Visual Studio Code

- **GUI DEVELOPMENT**

The GUI consists of two primary sections:

- **Detail Frame:** Contains input fields for entering student details.
- **Data Frame:** Displays the student data in a tabular format using a Treeview widget.

- **DATABASE INTERACTIONS**

- Database operations are handled using the pymysql library. SQL queries facilitate data retrieval, insertion, update, and deletion. Error handling ensures robustness, with feedback provided to users through message boxes.

RESULTS AND DISCUSSION

The SRMS successfully handles student data and performs all intended functionalities. The use of Tkinter enables a user-friendly interface, while MySQL ensures efficient and reliable data storage. During testing, the system demonstrated the following performance characteristics:

- **Efficiency:** Quick execution of queries for large datasets.
- **Ease of Use:** Intuitive GUI design allows for seamless operation.
- **Scalability:** Can be expanded to include additional features such as report generation and role-based access.

CONCLUSION AND FUTURE WORK

This laboratory exercise presents a functional SRMS designed using Python and MySQL. The system demonstrates the potential for streamlining student data management in academic settings. Future enhancements could include:

1. Integration of user authentication for data security.
2. Adding export features for generating reports in formats such as PDF or Excel.
3. Extending the system for web-based or mobile-based platforms.

REFERENCES

Udemy. (2024). *MySQL for beginners - Learn MySQL for free*. Udemy.
<https://www.udemy.com/course/mysql-for-beginners/>

W3Schools. (n.d.). *Python MySQL tutorial*. W3Schools.
https://www.w3schools.com/python/python_mysql_getstarted.asp